

YouTube Video Summarizer ❓❓❓ Project Documentation

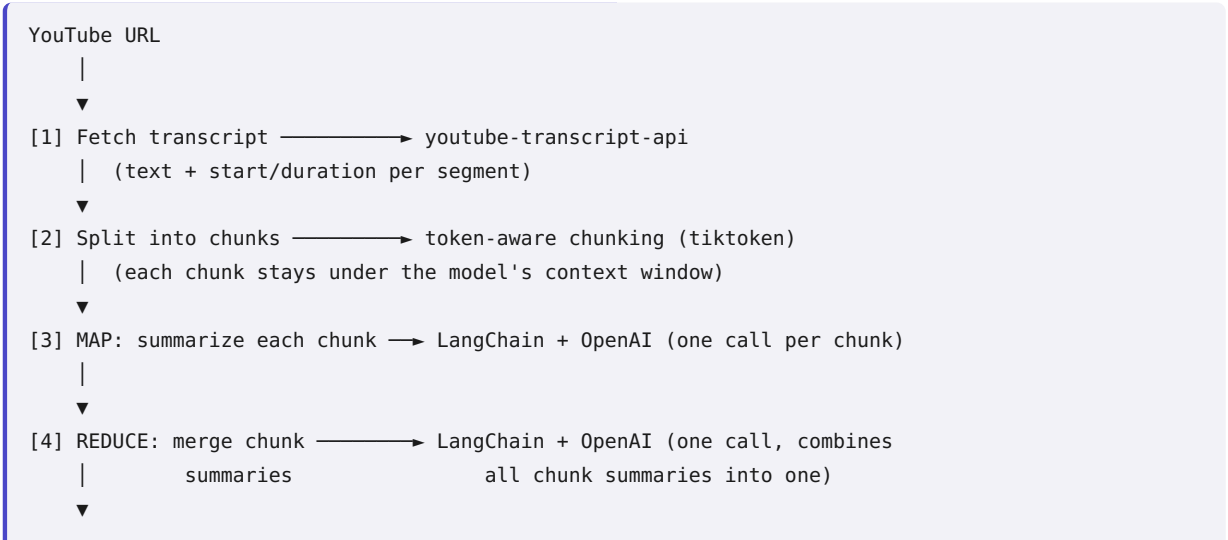
- YouTube Video Summarizer
 - 1. What you’re building
 - 2. Project structure
 - 3. Prerequisites
 - 4. Setup
 - 5. Running the app
 - 6. How each build step works
 - 7. Configuration knobs (in the Streamlit sidebar)
 - 8. Cost & performance notes
 - 9. Known limitations
 - 10. Possible extensions
 - 11. Troubleshooting
 - 12. Deploying beyond localhost

YouTube Video Summarizer

An NLP app that takes any YouTube URL, fetches its transcript, and produces a clean summary + timestamped chapters — even for videos long enough to blow past a single LLM call’s context window. It uses a **map-reduce** summarization pattern: each chunk of transcript is summarized on its own (map), then all chunk summaries are merged into one cohesive final summary (reduce).

Category	NLP
Difficulty	Beginner
Stack	youtube-transcript-api , LangChain, OpenAI, Streamlit
Interface	Streamlit web app

1. What you’re building



[5] Deploy as Streamlit app → final summary + clickable chapters
with chapter timestamps with jump-to-timestamp links

Why map-reduce instead of “just paste the whole transcript into the prompt”? A 2-hour podcast transcript can be 25,000+ words — more tokens than many models’ context windows once you add prompt instructions and leave room for the response. Map-reduce keeps every individual LLM call small and bounded, no matter how long the source video is.

2. Project structure

```
youtube_summarizer/
├─ app.py                # Streamlit UI – wires all steps together
├─ requirements.txt      # Python dependencies
├─ .env.example          # Template for your OpenAI API key
├─ src/
│   └─ __init__.py
│   └─ transcript_fetcher.py # Step 1: fetch transcript + video ID parsing
│   └─ chunker.py          # Step 2: token-aware chunking
│   └─ summarizer.py       # Steps 3 & 4: map-reduce summarization
│   └─ chapters.py         # Builds timestamped chapter list
```

3. Prerequisites

- Python 3.9+
- An OpenAI API key (platform.openai.com)
- A YouTube video that **has captions** (auto-generated captions work fine; videos with captions fully disabled will fail — this is a YouTube-side limitation, not something the app can work around)

4. Setup

```
# 1. Create the project folder and enter it
mkdir youtube_summarizer && cd youtube_summarizer

# 2. Create a virtual environment
python3 -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Configure your API key
cp .env.example .env
# then open .env and paste your real OpenAI key
```

requirements.txt :

```
youtube-transcript-api==0.6.2
langchain==0.2.14
langchain-openai==0.1.22
langchain-community==0.2.12
openai==1.42.0
```

```
streamlit==1.37.1
tiktoken==0.7.0
python-dotenv==1.0.1
```

5. Running the app

```
streamlit run app.py
```

This opens a browser tab (usually `http://localhost:8501`). From there:

1. Paste your OpenAI API key in the sidebar (or rely on `.env`).
2. Pick a model — `gpt-4o-mini` is the cheapest/fastest and is the default.
3. Paste a YouTube URL (works with `watch?v=`, `youtu.be/`, `/embed/`, and `/shorts/` links).
4. Click **Summarize**.

You'll see, in order: the embedded video, a transcript-fetched confirmation, a chunk count, the final summary, and an expandable list of timestamped chapters — each with a “jump to this point in the video” link.

6. How each build step works

Step 1 — Fetch transcript (`src/transcript_fetcher.py`)

- `extract_video_id()` pulls the 11-character video ID out of any common YouTube URL shape using regex, so users can paste whatever link format they have.
- `fetch_transcript()` calls `YouTubeTranscriptApi.get_transcript()`, preferring English but falling back to *any* available transcript (e.g., auto-generated captions in another language) if English isn't found — so the app degrades gracefully instead of just failing.
- Each transcript entry comes back as `{text, start, duration}`. We keep this structure (rather than flattening immediately) because the `start` timestamps are exactly what let us build chapter markers later.
- Common failure modes handled explicitly: `TranscriptsDisabled` (captions turned off), `NoTranscriptFound`, `VideoUnavailable` (private/deleted video) — each surfaces a clear error message in the UI instead of a raw stack trace.

Step 2 — Split into chunks (`src/chunker.py`)

- Uses `tiktoken` to count **actual tokens**, not characters or words — this matters because token density varies a lot by language and by how “chatty” a transcript is.
- Greedily packs consecutive transcript segments into a chunk until adding the next segment would exceed `max_tokens` (default: 1800 — comfortably under any modern model's window, leaving headroom for the prompt template and the response).
- Each `TranscriptChunk` remembers its own `start_time` and `end_time`, so later steps can say *when* in the video a given piece of content happened.

Steps 3 & 4 — Map-reduce summarization (`src/summarizer.py`)

- **Map:** `map_summarize_chunks()` runs one LLM call per chunk using a prompt that asks for a focused 3–5 sentence summary of *just that chunk*, with no reference to “chunks” or “segments” in the output.
- **Reduce:** `reduce_summaries()` takes all the chunk summaries (each labeled with its timestamp) and makes **one final LLM call** that merges them into a single video-level summary: a short overview paragraph plus a bulleted “Key Points” section, written as if the model watched the whole video start to finish.

- This two-phase structure is what makes the whole pipeline scale: no matter how many chunks a video produces, the reduce step only ever processes summaries (which are short), not raw transcript text.

Step 5 — Chapters + Streamlit UI (`src/chapters.py` , `app.py`)

- `build_chapters()` takes the same per-chunk summaries from the map step and (optionally) makes one more small LLM call per chunk to generate a short, punchy chapter title (≤ 6 words). This can be turned off in the sidebar to save API calls — it then falls back to using the first few words of the chunk summary as the title.
- `app.py` renders:
 - The embedded YouTube video
 - Live status messages as each pipeline step runs (fetch $\rightarrow$ chunk $\rightarrow$ map $\rightarrow$ reduce $\rightarrow$ chapters), via `st.spinner`
 - The final merged summary
 - An expandable list of chapters, each showing its timestamp, title, a “jump to this point” link (`&t=<seconds>s`), and its summary text
 - A collapsible “raw transcript” section for reference

7. Configuration knobs (in the Streamlit sidebar)

Setting	What it controls
OpenAI API Key	Your key; can also be set via <code>.env</code>
Model	<code>gpt-4o-mini</code> (cheap/fast, default), <code>gpt-4o</code> , <code>gpt-4.1-mini</code> , <code>gpt-4.1</code>
Max tokens per chunk	Trade-off: smaller chunks = more, cheaper calls with tighter focus; larger chunks = fewer calls but more context per call
Generate AI chapter titles	Toggle off to skip the extra per-chapter LLM call and reduce cost/latency

8. Cost & performance notes

- Cost scales with number of chunks: each chunk costs one “map” call, plus one “reduce” call total, plus one “chapter title” call per chunk if enabled. A 20-minute video might produce 2–4 chunks; a 3-hour video might produce 15–20.
- `gpt-4o-mini` is recommended for cost — it’s inexpensive and handles summarization well. Switch to `gpt-4o` for noticeably higher-quality summaries on complex/technical content.
- Transcript fetching itself is free and fast (no OpenAI cost) — it only hits YouTube’s caption endpoint.

9. Known limitations

- Videos with captions fully disabled cannot be summarized (no transcript exists to fetch) — this is inherent to relying on YouTube captions.
- Auto-generated captions can contain transcription errors (especially with jargon, names, or heavy accents), which will carry through into the summary.
- This app doesn’t do speaker diarization — if multiple speakers talk, the transcript (and therefore the summary) won’t distinguish “who said what.”
- Chapter boundaries are based on chunk boundaries (driven by token count), not by actual topic changes — they’re a reasonable approximation, not scene-detection.

10. Possible extensions

- **True topic-based chaptering:** detect topic shifts in the transcript (e.g., via embeddings + clustering) instead of chunking purely by token count, so chapters align with actual content changes.
- **Caching:** store fetched transcripts/summaries (e.g., keyed by video ID) so re-summarizing the same video doesn't refetch or re-call the LLM.
- **Multi-language output:** ask the reduce step to output the summary in a language different from the transcript's language.
- **Export:** add a "Download summary as Markdown/PDF" button.
- **Batch mode:** accept a playlist URL and summarize every video in it.

11. Troubleshooting

Symptom	Likely cause	Fix
"Transcripts are disabled for this video"	Creator disabled captions	Try a different video
"No transcript found in ['en', ...]"	No English captions, and fallback also failed	Video may have zero captions of any kind
"This video is unavailable or private"	Deleted/private/region-locked video	Use a public video
Streamlit says missing API key	Key not set in sidebar or <code>.env</code>	Paste key in sidebar, or fill <code>.env</code> and restart the app
Slow on long videos	Many chunks = many sequential LLM calls	Increase "max tokens per chunk" to reduce chunk count, or disable AI chapter titles

12. Deploying beyond localhost

Once it works locally, you can deploy this Streamlit app for free on [Streamlit Community Cloud](#):

1. Push this project to a public (or private, with Cloud access) GitHub repo.
2. On Streamlit Community Cloud, click "New app," point it at your repo and `app.py`.
3. Add `OPENAI_API_KEY` under the app's **Secrets** settings instead of committing your `.env` file (never commit real API keys to GitHub).